

Seance 3

Classes Abstraites et Interfaces
Concevoir des systemes extensibles et flexibles

Cours de renforcement C++

Table des matières

1	Introduction : Du concret vers l'abstrait	3
1.1	Rappel de la seance precedente	3
1.2	La solution : Les classes abstraites	3
2	Les fonctions virtuelles pures	5
2.1	La syntaxe	5
2.2	Classe abstraite vs classe concrete	5
2.3	Pourquoi utiliser des fonctions virtuelles pures ?	6
3	Les interfaces en C++	7
3.1	Qu'est-ce qu'une interface ?	7
3.2	Pourquoi utiliser des interfaces ?	7
4	Exemples concrets d'interfaces	10
4.1	Interface de stockage	10
4.2	Interface de rendu	11
4.3	Interface de plugin	12
5	Heritage multiple d'interfaces	15
5.1	Le concept	15
5.2	Exemple : Peripheriques	15
6	Concevoir de bonnes interfaces	18
6.1	Principe 1 : Petites et cibles	18
6.2	Principe 2 : Noms clairs et descriptifs	18
6.3	Principe 3 : Pas de details d'implementation	19
6.4	Principe 4 : Methodes coherentes	19
7	Exercices pratiques	21
7.1	Exercice 1 : Interface de serialisation (facile)	21
7.2	Exercice 2 : Systeme de notification (moyen)	22
7.3	Exercice 3 : Cache avec strategies (difficile)	23
8	Patterns avec interfaces	26
8.1	Strategy Pattern	26
8.2	Observer Pattern	26
8.3	Factory Pattern	27

9	Pieges et bonnes pratiques	28
9.1	Piege 1 : Trop d'interfaces	28
9.2	Piege 2 : Oublier le destructeur virtuel	28
9.3	Bonne pratique : Utiliser smart pointers	28
10	Quiz de validation	30
10.1	Questions	30
10.2	Reponses	30
11	Projet : Systeme de plugins modulaire	31
11.1	Objectif	31
11.2	Code de base	31
11.3	Extensions possibles	32

1 Introduction : Du concret vers l'abstrait

1.1 Rappel de la seance precedente

Dans la Seance 2, nous avons vu comment creer une hierarchie de classes avec l'heritage et le polymorphisme :

```

1 class Animal {
2 public:
3     virtual void parler() {
4         std::cout << "...\\n"; // Implementation par default
5     }
6     virtual ~Animal() = default;
7 };
8
9 class Chien : public Animal {
10     void parler() override {
11         std::cout << "Ouaf !\\n"; // Implementation specifique
12     }
13 };

```

Probleme : Que se passe-t-il si on cree un objet `Animal` directement ?

```

1 Animal a;
2 a.parler(); // Affiche : ...
3 // Ca marche, mais ca n'a pas vraiment de sens !
4 // Un "Animal" generique, ca n'existe pas vraiment...

```

La question philosophique

Qu'est-ce qu'un "Animal" ?

Dans la vraie vie, il n'existe pas d' "animal" tout court. Il existe des chiens, des chats, des oiseaux, etc. "Animal" est un concept abstrait, une categorie.

En programmation : On veut souvent definir des concepts generaux (Animal, Forme, Vehicule) sans qu'on puisse les instancier directement.

1.2 La solution : Les classes abstraites

Une classe abstraite est une classe qu'on ne peut PAS instancier.

Elle sert uniquement de modele, de contrat, de "squelette" pour d'autres classes.

```

1 class Animal {
2 public:
3     // Fonction virtuelle PURE : pas d'implementation
4     virtual void parler() = 0; // Le "= 0" rend la fonction pure
5     virtual ~Animal() = default;
6 };
7
8 // Animal a; // ERREUR de compilation !
9 // "Cannot instantiate abstract class"
10
11 class Chien : public Animal {
12     void parler() override {
13         std::cout << "Ouaf !\\n";
14     }
15 };
16
17 Chien rex; // OK ! Chien est concret

```

Les avantages

- **Clarte conceptuelle** : “Animal” est clairement un concept, pas quelque chose de concret
- **Force les derivees** : Toute classe qui herite d’Animal DOIT implementer parler()
- **Securite** : Impossible de creer accidentellement un Animal generique
- **Documentation** : La classe abstraite documente ce que TOUTES les classes derivees doivent faire

2 Les fonctions virtuelles pures

2.1 La syntaxe

```

1 class MaClasse {
2 public:
3     virtual void maFonction() = 0; // Virtuelle pure
4 };

```

Decomposition

virtual : Cette fonction peut etre redefinie (polymorphisme)

= 0 : Cette fonction N'A PAS d'implementation dans cette classe. Elle est "pure".

Consequence : La classe devient abstraite. On ne peut plus l'instancier.

2.2 Classe abstraite vs classe concrete

```

1 // Classe ABSTRAITE : au moins une fonction virtuelle pure
2 class Forme {
3 public:
4     virtual double aire() const = 0;           // Pure
5     virtual double perimetre() const = 0;      // Pure
6     virtual ~Forme() = default;
7
8     // On peut avoir des fonctions normales aussi
9     void afficher() const {
10         std::cout << "Aire : " << aire() << "\n";
11         std::cout << "Perimetre : " << perimetre() << "\n";
12     }
13 };
14
15 // Forme f; // ERREUR ! Forme est abstraite
16
17 // Classe CONCRETE : toutes les fonctions sont implementees
18 class Cercle : public Forme {
19 private:
20     double rayon;
21
22 public:
23     Cercle(double r) : rayon(r) {}
24
25     double aire() const override {
26         return 3.14159 * rayon * rayon;
27     }
28
29     double perimetre() const override {
30         return 2 * 3.14159 * rayon;
31     }
32 };
33
34 Cercle c(5.0); // OK ! Cercle est concrete

```

Regle importante

Pour qu'une classe derivee soit concrete, elle **DOIT** implementer **TOUTES** les fonctions virtuelles pures de sa classe de base.

Si elle n'en implemente que certaines, elle reste abstraite !

```

1 class FormeIncomplete : public Forme {
2     double aire() const override {
3         return 0; // Implemente aire()
4     }
5     // Mais pas perimetre() !
6 };
7
8 // FormeIncomplete f; // ERREUR ! Reste abstraite

```

2.3 Pourquoi utiliser des fonctions virtuelles pures ?

Exemple concret : Un systeme de logging

Sans fonctions pures (mauvais) :

```

1 class Logger {
2 public:
3     virtual void log(const std::string& msg) {
4         // Oups, on a oublie d'implementer quelque chose !
5     }
6 };
7
8 class FileLogger : public Logger {
9     // On herite de la version vide... bug garanti !
10 };

```

Avec fonctions pures (bon) :

```

1 class Logger {
2 public:
3     virtual void log(const std::string& msg) = 0;
4     virtual ~Logger() = default;
5 };
6
7 class FileLogger : public Logger {
8     // OBLIGATION d'implementer log()
9     // Sinon : erreur de compilation !
10    void log(const std::string& msg) override {
11        // Implementation reelle
12    }
13 };

```

L'avantage

Le compilateur **FORCE** les classes derivees a implementer toutes les fonctions importantes.

Impossible d'oublier ! Le code ne compilera pas tant que tout n'est pas implemente.

C'est comme un contrat que toutes les classes derivees doivent respecter.

3 Les interfaces en C++

3.1 Qu'est-ce qu'une interface ?

En theorie : Une interface est une classe qui ne contient QUE des fonctions virtuelles pures. Pas de donnees membres, pas d'implementation.

En C++ : Il n'y a pas de mot-cle special `interface` comme en Java. On utilise simplement une classe abstraite pure.

```

1 // INTERFACE : que des fonctions virtuelles pures
2 class ILogger {
3 public:
4     virtual void log(const std::string& message) = 0;
5     virtual void warning(const std::string& message) = 0;
6     virtual void error(const std::string& message) = 0;
7     virtual ~ILogger() = default;
8
9     // PAS de donnees membres !
10    // PAS d'implementation !
11 };

```

Convention de nommage

Prefixe "I" : Par convention, on prefixe souvent les interfaces avec "I" (comme ILogger, IRenderer, IPlugin).

Ce n'est pas obligatoire, mais ca aide a reconnaitre immediatement qu'il s'agit d'une interface.

Alternatives : Certains preferent des noms comme `LoggerInterface` ou simplement `Logger`.

3.2 Pourquoi utiliser des interfaces ?

1. Decoupler le code

Imaginez que vous ecrivez une application. Vous voulez logger des messages, mais vous ne savez pas encore si ca ira dans un fichier, dans la console, ou sur le reseau.

Sans interface (mauvais) :

```

1 class Application {
2 private:
3     FileLogger logger; // Couple a FileLogger !
4
5 public:
6     void faireTravail() {
7         logger.log("Travail commence");
8         // ... travail ...
9         logger.log("Travail termine");
10    }
11 };
12
13 // Si on veut changer pour ConsoleLogger, il faut modifier Application !

```

Avec interface (bon) :

```

1 class Application {
2 private:
3     ILogger* logger; // Depend de l'interface, pas de l'implementation !
4

```

```

5 public:
6     Application(ILogger* l) : logger(l) {}
7
8     void faireTravail() {
9         logger->log("Travail commence");
10        // ... travail ...
11        logger->log("Travail termine");
12    }
13 };
14
15 // Utilisation :
16 FileLogger fileLog;
17 Application app1(&fileLog);
18
19 ConsoleLogger consoleLog;
20 Application app2(&consoleLog);
21
22 // On peut changer le logger sans toucher a Application !

```

Le principe

Programmer vers une interface, pas vers une implementation.

Application ne sait pas (et ne veut pas savoir) quel type de logger elle utilise. Elle sait juste qu'il respecte l'interface ILogger.

Flexibilité maximale : On peut changer le logger a tout moment sans modifier Application.

2. Tester facilement

Les interfaces rendent les tests beaucoup plus faciles :

```

1 // Logger de test : ne fait rien, ou stocke les messages
2 class MockLogger : public ILogger {
3 private:
4     std::vector<std::string> messages;
5
6 public:
7     void log(const std::string& msg) override {
8         messages.push_back(msg);
9     }
10
11     void warning(const std::string& msg) override {
12         messages.push_back("WARN: " + msg);
13     }
14
15     void error(const std::string& msg) override {
16         messages.push_back("ERROR: " + msg);
17     }
18
19     // Pour les tests
20     size_t count() const { return messages.size(); }
21     std::string get(size_t i) const { return messages[i]; }
22 };
23
24 // Dans les tests :
25 void testApplication() {
26     MockLogger mockLog;
27     Application app(&mockLog);

```

```

28
29     app.faireTravail();
30
31     // Verifier que les bons messages ont ete logges
32     assert(mockLog.count() == 2);
33     assert(mockLog.get(0) == "Travail commence");
34     assert(mockLog.get(1) == "Travail termine");
35 }

```

3. Permettre plusieurs implementations

```

1  class ConsoleLogger : public ILogger {
2  public:
3      void log(const std::string& msg) override {
4          std::cout << "[LOG] " << msg << "\n";
5      }
6      void warning(const std::string& msg) override {
7          std::cout << "[WARN] " << msg << "\n";
8      }
9      void error(const std::string& msg) override {
10         std::cerr << "[ERROR] " << msg << "\n";
11     }
12 };
13
14 class FileLogger : public ILogger {
15 private:
16     std::ofstream file;
17 public:
18     FileLogger(const std::string& filename) {
19         file.open(filename, std::ios::app);
20     }
21     void log(const std::string& msg) override {
22         file << "[LOG] " << msg << "\n";
23     }
24     void warning(const std::string& msg) override {
25         file << "[WARN] " << msg << "\n";
26     }
27     void error(const std::string& msg) override {
28         file << "[ERROR] " << msg << "\n";
29     }
30 };
31
32 class NetworkLogger : public ILogger {
33 public:
34     void log(const std::string& msg) override {
35         // Envoyer sur le reseau...
36     }
37     void warning(const std::string& msg) override {
38         // Envoyer sur le reseau...
39     }
40     void error(const std::string& msg) override {
41         // Envoyer sur le reseau...
42     }
43 };

```

4 Exemples concrets d'interfaces

4.1 Interface de stockage

```
1 class IStorage {
2 public:
3     virtual void save(const std::string& key,
4                       const std::string& value) = 0;
5     virtual std::string load(const std::string& key) = 0;
6     virtual bool exists(const std::string& key) const = 0;
7     virtual void remove(const std::string& key) = 0;
8     virtual ~IStorage() = default;
9 };
10
11 // Implementation en memoire
12 class MemoryStorage : public IStorage {
13 private:
14     std::map<std::string, std::string> data;
15
16 public:
17     void save(const std::string& key,
18              const std::string& value) override {
19         data[key] = value;
20     }
21
22     std::string load(const std::string& key) override {
23         if (data.find(key) != data.end()) {
24             return data[key];
25         }
26         return "";
27     }
28
29     bool exists(const std::string& key) const override {
30         return data.find(key) != data.end();
31     }
32
33     void remove(const std::string& key) override {
34         data.erase(key);
35     }
36 };
37
38 // Implementation avec fichiers
39 class FileStorage : public IStorage {
40 private:
41     std::string directory;
42
43 public:
44     FileStorage(const std::string& dir) : directory(dir) {}
45
46     void save(const std::string& key,
47              const std::string& value) override {
48         std::ofstream file(directory + "/" + key);
49         file << value;
50     }
51
52     std::string load(const std::string& key) override {
53         std::ifstream file(directory + "/" + key);
54         std::string value;
```

```

55     std::getline(file, value);
56     return value;
57 }
58
59 bool exists(const std::string& key) const override {
60     std::ifstream file(directory + "/" + key);
61     return file.good();
62 }
63
64 void remove(const std::string& key) override {
65     std::remove((directory + "/" + key).c_str());
66 }
67 };

```

Utilisation :

```

1  class UserManager {
2  private:
3      IStorage* storage;
4
5  public:
6      UserManager(IStorage* s) : storage(s) {}
7
8      void saveUser(const std::string& username,
9                  const std::string& data) {
10         storage->save("user_" + username, data);
11     }
12
13     std::string getUser(const std::string& username) {
14         return storage->load("user_" + username);
15     }
16 };
17
18 int main() {
19     // En developpement : stockage en memoire (rapide)
20     MemoryStorage memStore;
21     UserManager devManager(&memStore);
22
23     // En production : stockage sur disque (persistant)
24     FileStorage fileStore("/data/users");
25     UserManager prodManager(&fileStore);
26
27     // Le code de UserManager est identique !
28 }

```

4.2 Interface de rendu

```

1  class IRenderer {
2  public:
3      virtual void init(int width, int height) = 0;
4      virtual void clear() = 0;
5      virtual void drawText(int x, int y,
6                          const std::string& text) = 0;
7      virtual void drawRect(int x, int y,
8                          int width, int height) = 0;
9      virtual void render() = 0;
10     virtual ~IRenderer() = default;
11 };

```

```

12
13 class ConsoleRenderer : public IRenderer {
14 private:
15     int width, height;
16     std::vector<std::string> buffer;
17
18 public:
19     void init(int w, int h) override {
20         width = w;
21         height = h;
22         buffer.resize(height, std::string(width, ' '));
23     }
24
25     void clear() override {
26         for (auto& line : buffer) {
27             line = std::string(width, ' ');
28         }
29     }
30
31     void drawText(int x, int y, const std::string& text) override {
32         if (y >= 0 && y < height) {
33             for (size_t i = 0; i < text.length() && x + i < width; ++i)
34                 buffer[y][x + i] = text[i];
35         }
36     }
37
38
39     void drawRect(int x, int y, int w, int h) override {
40         for (int i = 0; i < h && y + i < height; ++i) {
41             for (int j = 0; j < w && x + j < width; ++j) {
42                 buffer[y + i][x + j] = '#';
43             }
44         }
45     }
46
47     void render() override {
48         for (const auto& line : buffer) {
49             std::cout << line << "\n";
50         }
51     }
52 };
53
54 class SFMLRenderer : public IRenderer {
55     // Implementation avec SFML...
56     void init(int w, int h) override {
57         // Creer fenetre SFML...
58     }
59     // etc.
60 };

```

4.3 Interface de plugin

```

1 class IPlugin {
2 public:
3     virtual std::string name() const = 0;
4     virtual std::string version() const = 0;

```

```

5     virtual void init() = 0;
6     virtual void update() = 0;
7     virtual void shutdown() = 0;
8     virtual ~IPlugin() = default;
9 };
10
11 class CPUMonitor : public IPlugin {
12 public:
13     std::string name() const override {
14         return "CPU Monitor";
15     }
16
17     std::string version() const override {
18         return "1.0.0";
19     }
20
21     void init() override {
22         std::cout << "CPU Monitor initialise\n";
23     }
24
25     void update() override {
26         // Lire /proc/stat, calculer le pourcentage CPU...
27     }
28
29     void shutdown() override {
30         std::cout << "CPU Monitor arrete\n";
31     }
32 };
33
34 class RAMMonitor : public IPlugin {
35 public:
36     std::string name() const override {
37         return "RAM Monitor";
38     }
39
40     std::string version() const override {
41         return "1.0.0";
42     }
43
44     void init() override {
45         std::cout << "RAM Monitor initialise\n";
46     }
47
48     void update() override {
49         // Lire /proc/meminfo...
50     }
51
52     void shutdown() override {
53         std::cout << "RAM Monitor arrete\n";
54     }
55 };
56
57 class PluginManager {
58 private:
59     std::vector<IPlugin*> plugins;
60
61 public:
62     void addPlugin(IPlugin* p) {

```

```
63     plugins.push_back(p);
64 }
65
66 void initAll() {
67     for (auto* p : plugins) {
68         std::cout << "Initialisation de " << p->name()
69                 << " v" << p->version() << "\n";
70         p->init();
71     }
72 }
73
74 void updateAll() {
75     for (auto* p : plugins) {
76         p->update();
77     }
78 }
79
80 void shutdownAll() {
81     for (auto* p : plugins) {
82         p->shutdown();
83     }
84 }
85 };
```

5 Héritage multiple d'interfaces

5.1 Le concept

En C++, une classe peut hériter de plusieurs interfaces :

```

1  class IReadable {
2  public:
3      virtual std::string read() = 0;
4      virtual ~IReadable() = default;
5  };
6
7  class IWritable {
8  public:
9      virtual void write(const std::string& data) = 0;
10     virtual ~IWritable() = default;
11 };
12
13 // Une classe peut implementer les deux interfaces
14 class File : public IReadable, public IWritable {
15 private:
16     std::string filename;
17     std::string content;
18
19 public:
20     File(const std::string& name) : filename(name) {}
21
22     std::string read() override {
23         std::ifstream f(filename);
24         std::string line, result;
25         while (std::getline(f, line)) {
26             result += line + "\n";
27         }
28         return result;
29     }
30
31     void write(const std::string& data) override {
32         std::ofstream f(filename);
33         f << data;
34     }
35 };

```

Syntaxe

```
class File : public IReadable, public IWritable
```

Séparez les interfaces avec des virgules. L'ordre n'a pas d'importance.

5.2 Exemple : Peripheriques

```

1  class IPrintable {
2  public:
3      virtual void print(const std::string& document) = 0;
4      virtual ~IPrintable() = default;
5  };
6
7  class IScannable {
8  public:

```

```

9     virtual std::string scan() = 0;
10    virtual ~IScannable() = default;
11 };
12
13 class IFaxable {
14 public:
15     virtual void sendFax(const std::string& number,
16                          const std::string& document) = 0;
17     virtual ~IFaxable() = default;
18 };
19
20 // Une imprimante simple
21 class Printer : public IPrintable {
22 public:
23     void print(const std::string& doc) override {
24         std::cout << "Impression : " << doc << "\n";
25     }
26 };
27
28 // Un scanner simple
29 class Scanner : public IScannable {
30 public:
31     std::string scan() override {
32         return "Document scanne";
33     }
34 };
35
36 // Une imprimante multifonction
37 class AllInOne : public IPrintable, public IScannable, public IFaxable {
38 public:
39     void print(const std::string& doc) override {
40         std::cout << "Impression : " << doc << "\n";
41     }
42
43     std::string scan() override {
44         return "Document scanne";
45     }
46
47     void sendFax(const std::string& number,
48                 const std::string& doc) override {
49         std::cout << "Envoi fax au " << number << "\n";
50     }
51 };

```

Utilisation :

```

1 void faireTravail(IPrintable* printer, IScannable* scanner) {
2     std::string doc = scanner->scan();
3     printer->print(doc);
4 }
5
6 int main() {
7     AllInOne multifonction;
8
9     // On peut utiliser multifonction comme IPrintable
10    IPrintable* p = &multifonction;
11    p->print("Document");
12
13    // Ou comme IScannable

```

```
14     IScannable* s = &multifonction;  
15     s->scan();  
16  
17     // Ou les deux !  
18     faireTravail(&multifonction, &multifonction);  
19 }
```

L'avantage

Une classe peut jouer plusieurs roles !

AllInOne peut etre utilisee partout ou on attend un IPrintable, un IScannable, ou un IFaxable.

Principe de segregation d'interface : Il vaut mieux plusieurs petites interfaces specifiques qu'une grosse interface generale.

6 Concevoir de bonnes interfaces

6.1 Principe 1 : Petites et ciblées

Mauvais : Interface trop grosse

```

1 class IWorker {
2 public:
3     virtual void work() = 0;
4     virtual void eat() = 0;
5     virtual void sleep() = 0;
6     virtual void managePeople() = 0; // Tous ne sont pas managers !
7     virtual void code() = 0;         // Tous ne sont pas devs !
8     virtual ~IWorker() = default;
9 };

```

Bon : Interfaces separees

```

1 class IWorker {
2 public:
3     virtual void work() = 0;
4     virtual ~IWorker() = default;
5 };
6
7 class IManager {
8 public:
9     virtual void managePeople() = 0;
10    virtual ~IManager() = default;
11 };
12
13 class IDeveloper {
14 public:
15     virtual void code() = 0;
16     virtual ~IDeveloper() = default;
17 };
18
19 // Un dev normal
20 class Developer : public IWorker, public IDeveloper {
21     // ...
22 };
23
24 // Un lead dev
25 class LeadDeveloper : public IWorker, public IDeveloper, public
    IManager {
26     // ...
27 };

```

6.2 Principe 2 : Noms clairs et descriptifs

```

1 // Mauvais noms
2 class IData { /* ... */ };
3 class IThing { /* ... */ };
4 class IStuff { /* ... */ };
5
6 // Bons noms

```

```

7 class ISerializable { /* ... */ }; // Clair : peut etre serialise
8 class IComparable { /* ... */ }; // Clair : peut etre compare
9 class IDisposable { /* ... */ }; // Clair : peut etre dispose

```

6.3 Principe 3 : Pas de details d'implementation

Mauvais : Expose l'implementation

```

1 class IDatabase {
2 public:
3     virtual void openSQLConnection() = 0;
4     virtual void executeSQLQuery(const std::string& sql) = 0;
5     // <- Suppose que c'est une base SQL !
6 };

```

Bon : Interface abstraite

```

1 class IDatabase {
2 public:
3     virtual void connect() = 0;
4     virtual std::vector<Record> query(const Query& q) = 0;
5     // <- Pas de details sur le type de base
6 };
7
8 // Peut etre implemente avec SQL, NoSQL, fichiers, etc.

```

6.4 Principe 4 : Methodes coherentes

Toutes les methodes d'une interface doivent avoir un rapport logique entre elles.

Mauvais : Melange de concepts

```

1 class IUser {
2 public:
3     virtual std::string getName() = 0;
4     virtual void sendEmail(const std::string& msg) = 0;
5     virtual void calculateTaxes() = 0;
6     virtual void renderHTML() = 0;
7     // <- Aucune coherence !
8 };

```

Bon : Une responsabilite par interface

```
1 class IUser {
2 public:
3     virtual std::string getName() = 0;
4     virtual std::string getEmail() = 0;
5 };
6
7 class INotifiable {
8 public:
9     virtual void sendNotification(const std::string& msg) = 0;
10 };
11
12 class ITaxable {
13 public:
14     virtual double calculateTax() = 0;
15 };
```

7 Exercices pratiques

7.1 Exercice 1 : Interface de serialisation (facile)

Enonce

Creez une interface ISerializable avec deux methodes :

- `serialize()` : convertit l'objet en string
- `deserialize(string)` : reconstruit l'objet depuis une string

Implementez cette interface pour une classe `Point` avec `x` et `y`.

Solution :

```
1 class ISerializable {
2 public:
3     virtual std::string serialize() const = 0;
4     virtual void deserialize(const std::string& data) = 0;
5     virtual ~ISerializable() = default;
6 };
7
8 class Point : public ISerializable {
9 private:
10     int x, y;
11
12 public:
13     Point(int px = 0, int py = 0) : x(px), y(py) {}
14
15     std::string serialize() const override {
16         return std::to_string(x) + "," + std::to_string(y);
17     }
18
19     void deserialize(const std::string& data) override {
20         size_t pos = data.find(',');
21         x = std::stoi(data.substr(0, pos));
22         y = std::stoi(data.substr(pos + 1));
23     }
24
25     void afficher() const {
26         std::cout << "(" << x << ", " << y << ")\n";
27     }
28 };
29
30 int main() {
31     Point p1(10, 20);
32     std::string data = p1.serialize();
33     std::cout << "Serialise : " << data << "\n"; // 10,20
34
35     Point p2;
36     p2.deserialize(data);
37     p2.afficher(); // (10, 20)
38 }
```

7.2 Exercice 2 : Systeme de notification (moyen)

Enonce

Creez une interface `INotifier` avec une methode `notify(message)`.

Implementez trois classes :

- `EmailNotifier` : affiche "Email envoye : [message]"
- `SMSNotifier` : affiche "SMS envoye : [message]"
- `PushNotifier` : affiche "Notification push : [message]"

Creez une classe `NotificationManager` qui peut envoyer via plusieurs notifiers.

Solution :

```

1 class INotifier {
2 public:
3     virtual void notify(const std::string& message) = 0;
4     virtual ~INotifier() = default;
5 };
6
7 class EmailNotifier : public INotifier {
8 public:
9     void notify(const std::string& msg) override {
10         std::cout << "Email envoye : " << msg << "\n";
11     }
12 };
13
14 class SMSNotifier : public INotifier {
15 public:
16     void notify(const std::string& msg) override {
17         std::cout << "SMS envoye : " << msg << "\n";
18     }
19 };
20
21 class PushNotifier : public INotifier {
22 public:
23     void notify(const std::string& msg) override {
24         std::cout << "Notification push : " << msg << "\n";
25     }
26 };
27
28 class NotificationManager {
29 private:
30     std::vector<INotifier*> notifiers;
31
32 public:
33     void addNotifier(INotifier* n) {
34         notifiers.push_back(n);
35     }
36
37     void sendToAll(const std::string& message) {
38         std::cout << "Envoi de : \"" << message << "\"\n";
39         for (auto* n : notifiers) {
40             n->notify(message);
41         }
42     }
43 };
44
```

```

45 int main() {
46     EmailNotifier email;
47     SMSNotifier sms;
48     PushNotifier push;
49
50     NotificationManager manager;
51     manager.addNotifier(&email);
52     manager.addNotifier(&sms);
53     manager.addNotifier(&push);
54
55     manager.sendToAll("Votre commande est prete !");
56 }

```

7.3 Exercice 3 : Cache avec strategies (difficile)

Enonce

Creez une interface ICacheStrategy avec :

- shouldEvict(key) : retourne true si la cle doit etre evinee
- onAccess(key) : appelee quand une cle est accedee
- onInsert(key) : appelee quand une cle est inseree

Implementez deux strategies :

- LRUStrategy : Evince le moins recemment utilise
- LFUStrategy : Evince le moins frequemment utilise

Creez une classe Cache qui utilise une strategie.

Solution :

```

1  class ICacheStrategy {
2  public:
3      virtual bool shouldEvict(const std::string& key) = 0;
4      virtual void onAccess(const std::string& key) = 0;
5      virtual void onInsert(const std::string& key) = 0;
6      virtual std::string selectVictim() = 0; // Cle a evincer
7      virtual ~ICacheStrategy() = default;
8  };
9
10 class LRUStrategy : public ICacheStrategy {
11 private:
12     std::map<std::string, int> lastAccess;
13     int timestamp;
14
15 public:
16     LRUStrategy() : timestamp(0) {}
17
18     bool shouldEvict(const std::string& key) override {
19         return true; // Pour simplifier
20     }
21
22     void onAccess(const std::string& key) override {
23         lastAccess[key] = ++timestamp;
24     }
25
26     void onInsert(const std::string& key) override {
27         lastAccess[key] = ++timestamp;

```

```

28     }
29
30     std::string selectVictim() override {
31         if (lastAccess.empty()) return "";
32
33         auto oldest = lastAccess.begin();
34         for (auto it = lastAccess.begin(); it != lastAccess.end(); ++it)
35             if (it->second < oldest->second) {
36                 oldest = it;
37             }
38     }
39     return oldest->first;
40 }
41 };
42
43 class LFUStrategy : public ICacheStrategy {
44 private:
45     std::map<std::string, int> frequency;
46
47 public:
48     bool shouldEvict(const std::string& key) override {
49         return true;
50     }
51
52     void onAccess(const std::string& key) override {
53         frequency[key]++;
54     }
55
56     void onInsert(const std::string& key) override {
57         frequency[key] = 1;
58     }
59
60     std::string selectVictim() override {
61         if (frequency.empty()) return "";
62
63         auto least = frequency.begin();
64         for (auto it = frequency.begin(); it != frequency.end(); ++it) {
65             if (it->second < least->second) {
66                 least = it;
67             }
68         }
69         return least->first;
70     }
71 };
72
73 class Cache {
74 private:
75     std::map<std::string, std::string> data;
76     ICacheStrategy* strategy;
77     size_t maxSize;
78
79 public:
80     Cache(ICacheStrategy* s, size_t max)
81         : strategy(s), maxSize(max) {}
82
83     void put(const std::string& key, const std::string& value) {
84         if (data.size() >= maxSize && data.find(key) == data.end()) {

```

```
85         // Cache plein, evincer
86         std::string victim = strategy->selectVictim();
87         data.erase(victim);
88         std::cout << "Eviction de : " << victim << "\n";
89     }
90
91     data[key] = value;
92     strategy->onInsert(key);
93 }
94
95 std::string get(const std::string& key) {
96     if (data.find(key) != data.end()) {
97         strategy->onAccess(key);
98         return data[key];
99     }
100     return "";
101 }
102 };
103
104 int main() {
105     LRUStrategy lru;
106     Cache cache(&lru, 3);
107
108     cache.put("a", "valeur_a");
109     cache.put("b", "valeur_b");
110     cache.put("c", "valeur_c");
111
112     cache.get("a"); // a devient le plus recent
113
114     cache.put("d", "valeur_d"); // b est evince (LRU)
115
116     return 0;
117 }
```

8 Patterns avec interfaces

8.1 Strategy Pattern

L'interface definit une strategie, et on peut en changer a la volée.

```

1 class ISortStrategy {
2 public:
3     virtual void sort(std::vector<int>& data) = 0;
4     virtual ~ISortStrategy() = default;
5 };
6
7 class BubbleSort : public ISortStrategy {
8 public:
9     void sort(std::vector<int>& data) override {
10         // Implementation du tri a bulles
11         std::cout << "Tri a bulles\n";
12     }
13 };
14
15 class QuickSort : public ISortStrategy {
16 public:
17     void sort(std::vector<int>& data) override {
18         // Implementation du tri rapide
19         std::cout << "Tri rapide\n";
20     }
21 };
22
23 class Sorter {
24 private:
25     ISortStrategy* strategy;
26
27 public:
28     void setStrategy(ISortStrategy* s) {
29         strategy = s;
30     }
31
32     void sort(std::vector<int>& data) {
33         strategy->sort(data);
34     }
35 };

```

8.2 Observer Pattern

```

1 class IObserver {
2 public:
3     virtual void update(const std::string& event) = 0;
4     virtual ~IObserver() = default;
5 };
6
7 class Subject {
8 private:
9     std::vector<IObserver*> observers;
10
11 public:
12     void attach(IObserver* obs) {
13         observers.push_back(obs);
14     }

```

```
15     void notify(const std::string& event) {
16         for (auto* obs : observers) {
17             obs->update(event);
18         }
19     }
20 }
21 };
22
23 class ConcreteObserver : public IObserver {
24 private:
25     std::string name;
26
27 public:
28     ConcreteObserver(const std::string& n) : name(n) {}
29
30     void update(const std::string& event) override {
31         std::cout << name << " a reçu : " << event << "\n";
32     }
33 };
```

8.3 Factory Pattern

```
1 class IAnimal {
2 public:
3     virtual void parler() = 0;
4     virtual ~IAnimal() = default;
5 };
6
7 class Chien : public IAnimal {
8     void parler() override { std::cout << "Ouaf !\n"; }
9 };
10
11 class Chat : public IAnimal {
12     void parler() override { std::cout << "Miaou !\n"; }
13 };
14
15 class AnimalFactory {
16 public:
17     static IAnimal* create(const std::string& type) {
18         if (type == "chien") return new Chien();
19         if (type == "chat") return new Chat();
20         return nullptr;
21     }
22 };
23
24 int main() {
25     IAnimal* animal = AnimalFactory::create("chien");
26     animal->parler(); // Ouaf !
27     delete animal;
28 }
```

9 Pieges et bonnes pratiques

9.1 Piege 1 : Trop d'interfaces

Attention

Créer une interface pour chaque petite chose peut rendre le code trop complexe.

Règle : Créez une interface uniquement si :

1. Vous avez (ou prévoyez) plusieurs implementations
2. Vous voulez découpler deux parties du code
3. Vous voulez faciliter les tests

9.2 Piege 2 : Oublier le destructeur virtuel

```

1 class ILogger {
2 public:
3     virtual void log(const std::string& msg) = 0;
4     // Oubli du destructeur virtuel !
5 };
6
7 class FileLogger : public ILogger {
8 private:
9     std::ofstream* file;
10 public:
11     FileLogger() { file = new std::ofstream("log.txt"); }
12     ~FileLogger() { delete file; } // Ne sera jamais appele !
13     void log(const std::string& msg) override {
14         *file << msg << "\n";
15     }
16 };
17
18 ILogger* logger = new FileLogger();
19 delete logger; // Fuite memoire !

```

Solution

TOUJOURS mettre un destructeur virtuel dans les interfaces !

```

1 class ILogger {
2 public:
3     virtual void log(const std::string& msg) = 0;
4     virtual ~ILogger() = default; // <- OBLIGATOIRE
5 };

```

9.3 Bonne pratique : Utiliser smart pointers

```

1 class Application {
2 private:
3     std::unique_ptr<ILogger> logger;
4
5 public:
6     Application(std::unique_ptr<ILogger> l)
7         : logger(std::move(l)) {}
8

```

```
9     void run() {
10         logger->log("Application demarree");
11     }
12     // Pas besoin de delete : automatique !
13 };
14
15 int main() {
16     auto logger = std::make_unique<FileLogger>("app.log");
17     Application app(std::move(logger));
18     app.run();
19 }
```

10 Quiz de validation

10.1 Questions

1. Qu'est-ce qu'une fonction virtuelle pure ?
2. Quelle est la difference entre une classe abstraite et une interface ?
3. Peut-on instancier une classe avec une fonction virtuelle pure ?
4. Pourquoi utiliser des interfaces ?
5. Une classe peut-elle heriter de plusieurs interfaces ?
6. Faut-il toujours mettre un destructeur virtuel dans une interface ?
7. Qu'est-ce que le principe de segregation d'interface ?
8. Quelle est la convention de nommage pour les interfaces en C++ ?
9. Peut-on avoir des donnees membres dans une interface ?
10. Quelle est la difference entre `= 0` et `= default` ?

10.2 Reponses

1. Une fonction virtuelle pure est une fonction declaree avec `= 0`. Elle n'a pas d'implementation et rend la classe abstraite.
2. Une classe abstraite peut avoir des implementations. Une interface (en C++) n'a QUE des fonctions virtuelles pures.
3. Non ! Une classe avec au moins une fonction virtuelle pure est abstraite et ne peut pas etre instanciee.
4. Pour decouple le code, permettre plusieurs implementations, faciliter les tests, et rendre le code plus flexible.
5. Oui ! C'est meme recommande : plutot plusieurs petites interfaces qu'une grosse.
6. OUI ! Toujours. Sinon : fuites memoire quand on delete via un pointeur d'interface.
7. Il vaut mieux plusieurs petites interfaces specifiques qu'une grosse interface generale avec des methodes inutiles.
8. Prefixe "I" (ILogger, IRenderer). Mais ce n'est qu'une convention, pas une obligation.
9. Techniquement oui, mais c'est deconseille. Une interface pure ne devrait avoir que des methodes.
10. `= 0` : fonction virtuelle pure (pas d'implementation, classe abstraite). `= default` : utiliser l'implementation par default du compilateur.

11 Projet : Systeme de plugins modulaire

11.1 Objectif

Creer un systeme de plugins complet avec :

- Une interface IPlugin
- Un gestionnaire de plugins
- Plusieurs plugins concrets
- Chargement/dechargement dynamique

11.2 Code de base

```
1 class IPlugin {
2 public:
3     virtual std::string getName() const = 0;
4     virtual std::string getVersion() const = 0;
5     virtual std::string getDescription() const = 0;
6     virtual bool init() = 0;
7     virtual void execute() = 0;
8     virtual void shutdown() = 0;
9     virtual ~IPlugin() = default;
10 };
11
12 class PluginManager {
13 private:
14     std::map<std::string, std::unique_ptr<IPlugin>> plugins;
15
16 public:
17     void registerPlugin(std::unique_ptr<IPlugin> plugin) {
18         std::string name = plugin->getName();
19         if (plugin->init()) {
20             plugins[name] = std::move(plugin);
21             std::cout << "Plugin " << name << " charge\n";
22         } else {
23             std::cout << "Echec chargement " << name << "\n";
24         }
25     }
26
27     void executePlugin(const std::string& name) {
28         if (plugins.find(name) != plugins.end()) {
29             plugins[name]->execute();
30         }
31     }
32
33     void listPlugins() const {
34         std::cout << "=== Plugins charges ===\n";
35         for (const auto& pair : plugins) {
36             const auto& plugin = pair.second;
37             std::cout << "- " << plugin->getName()
38                 << " v" << plugin->getVersion() << "\n";
39             std::cout << " " << plugin->getDescription() << "\n";
40         }
41     }
42
43     ~PluginManager() {
44         for (auto& pair : plugins) {
```

```
45         pair.second->shutdown();
46     }
47 }
48 };
49
50 // Exemple de plugin
51 class HelloPlugin : public IPlugin {
52 public:
53     std::string getName() const override {
54         return "Hello";
55     }
56
57     std::string getVersion() const override {
58         return "1.0.0";
59     }
60
61     std::string getDescription() const override {
62         return "Plugin de demonstration";
63     }
64
65     bool init() override {
66         std::cout << "HelloPlugin initialise\n";
67         return true;
68     }
69
70     void execute() override {
71         std::cout << "Hello from plugin !\n";
72     }
73
74     void shutdown() override {
75         std::cout << "HelloPlugin arrete\n";
76     }
77 };
78
79 int main() {
80     PluginManager manager;
81
82     manager.registerPlugin(std::make_unique<HelloPlugin>());
83
84     manager.listPlugins();
85     manager.executePlugin("Hello");
86
87     return 0;
88 }
```

11.3 Extensions possibles

1. Ajouter d'autres plugins (Calculator, FileReader, etc.)
2. Ajouter un systeme de dependances entre plugins
3. Ajouter un systeme de configuration par plugin
4. Charger les plugins depuis des fichiers .so/.dll (avance)

Fin de la Seance 3

Prochaine seance : Smart Pointers et Gestion Mémoire